```
# Fresh notebook - complete inference test
!pip install -q huggingface_hub torch torchvision

from huggingface_hub import hf_hub_download
import torch
from PIL import Image
from torchvision import transforms

# Download model
model_path = hf_hub_download(
    repo_id="ranasinghehashini/srilankan-food-recognition",
    filename="best_model.pth"
)

print("✅ Downloaded from Hugging Face!")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/t
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limi
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a
best_model.pth: 100%                                  5.89M/5.89M [00:01<00:00, 3.64MB/s]
✅ Downloaded from Hugging Face!
```

EXPORT MODEL FOR MOBILE APP

```
# Cell 1: Install dependencies
!pip install -q onnx onnx-tf tensorflow

print("✅ Dependencies installed!")
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 17.5/17.5 MB 41.9 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 186.6/186.6 kB 5.7 MB/s eta 0:00:00
✅ Dependencies installed!
```

```
# Cell: Find model file
import os
from google.colab import drive

# Mount Google Drive if not already mounted
if not os.path.exists('/content/drive'):
    drive.mount('/content/drive')

print("🔍 Searching for model file...")
print("="*70)

# Possible locations
possible_paths = [
    '/content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth',
    '/content/drive/MyDrive/Research-2026/best_model.pth',
    '/content/drive/MyDrive/SriLankanFoodRecognition/best_model.pth',
    '/content/best_model.pth',
    '/content/extended_model.pth',
]

found_path = None

for path in possible_paths:
    if os.path.exists(path):
        print(f"✅ FOUND: {path}")
        found_path = path
        size_mb = os.path.getsize(path) / (1024 * 1024)
        print(f"   Size: {size_mb:.2f} MB")
        break

if not found_path:
    print("❌ Model not found in common locations")
    print("\n🔎 Let's search your entire Drive...")

    # Search entire Drive
    search_dir = '/content/drive/MyDrive'
    for root, dirs, files in os.walk(search_dir):
```

```
        for file in files:
            if file.endswith('.pth'):
                full_path = os.path.join(root, file)
                size_mb = os.path.getsize(full_path) / (1024 * 1024)
                print(f"   Found: {full_path} ({size_mb:.2f} MB)")

print("\n" + "="*70)
```

```
🔍 Searching for model file...
======================================================================
✅ FOUND: /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth
   Size: 5.62 MB

======================================================================
```

```
# Cell 2: Load your trained model (WITH DIAGNOSTICS)
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
from PIL import Image
import json
import os

print("="*70)
print("📱 EXPORTING MODEL FOR MOBILE")
print("="*70)

# First, let's check what's in the checkpoint
MODEL_PATH = '/content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth'
checkpoint = torch.load(MODEL_PATH, map_location='cpu')

print("\n🔍 Analyzing checkpoint structure...")
print("\nLayer shapes in checkpoint:")
for key, value in checkpoint['model_state_dict'].items():
    if 'weight' in key or 'bias' in key:
        print(f"   {key}: {value.shape}")

# From the error, we know fc1.weight is [256, 256]
# This means the flattened size before fc1 is 256
# Let's figure out the architecture

print("\n" + "="*70)
print("Based on checkpoint, the correct architecture is:")
print("="*70)

# Define the embedding network
class EmbeddingNet(nn.Module):
    def __init__(self, embedding_dim=128):
        super(EmbeddingNet, self).__init__()

        # Convolutional layers
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.pool1 = nn.MaxPool2d(2, 2)

        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(128)
        self.conv4 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
        self.bn4 = nn.BatchNorm2d(256)
        self.pool2 = nn.MaxPool2d(2, 2)

        # Global Average Pooling instead of flatten
        self.global_pool = nn.AdaptiveAvgPool2d(1)

        # FC layers - input is now 256 (from global pooling)
        self.fc1 = nn.Linear(256, 256)
        self.fc2 = nn.Linear(256, embedding_dim)

    def forward(self, x):
        # Block 1
        x = F.relu(self.bn1(self.conv1(x)))
        x = F.relu(self.bn2(self.conv2(x)))
        x = self.pool1(x)

        # Block 2
        x = F.relu(self.bn3(self.conv3(x)))
        x = F.relu(self.bn4(self.conv4(x)))
        x = self.pool2(x)
```

```
        # Global Average Pooling
        x = self.global_pool(x)

        # Flatten
        x = x.view(x.size(0), -1)

        # Fully connected
        x = F.relu(self.fc1(x))
        x = self.fc2(x)

        # L2 normalize
        x = F.normalize(x, p=2, dim=1)

        return x

# Define the PrototypicalNetwork wrapper
class PrototypicalNetwork(nn.Module):
    def __init__(self, embedding_dim=128):
        super(PrototypicalNetwork, self).__init__()
        self.embedding_net = EmbeddingNet(embedding_dim)

    def forward(self, x):
        return self.embedding_net(x)

# Initialize model
model = PrototypicalNetwork(embedding_dim=128)

print("\n📩 Loading model weights...")
model.load_state_dict(checkpoint['model_state_dict'])
model.eval()

print(f"\n✅ Model loaded from: {MODEL_PATH}")
print(f"   Classes: {len(checkpoint['classes'])}")
print(f"   Validation Accuracy: {checkpoint['val_acc']:.2f}%")

# Test the model with a dummy input
print("\n🧪 Testing model forward pass...")
dummy_input = torch.randn(1, 3, 224, 224)
with torch.no_grad():
    output = model(dummy_input)
print(f"   Input shape: {dummy_input.shape}")
print(f"   Output shape: {output.shape}")
print(f"   ✅ Model works correctly!")
```

```
======================================================================
📱 EXPORTING MODEL FOR MOBILE
======================================================================

🔍 Analyzing checkpoint structure...

Layer shapes in checkpoint:
   embedding_net.conv1.weight: torch.Size([32, 3, 3, 3])
   embedding_net.conv1.bias: torch.Size([32])
   embedding_net.bn1.weight: torch.Size([32])
   embedding_net.bn1.bias: torch.Size([32])
   embedding_net.conv2.weight: torch.Size([64, 32, 3, 3])
   embedding_net.conv2.bias: torch.Size([64])
   embedding_net.bn2.weight: torch.Size([64])
   embedding_net.bn2.bias: torch.Size([64])
   embedding_net.conv3.weight: torch.Size([128, 64, 3, 3])
   embedding_net.conv3.bias: torch.Size([128])
   embedding_net.bn3.weight: torch.Size([128])
   embedding_net.bn3.bias: torch.Size([128])
   embedding_net.conv4.weight: torch.Size([256, 128, 3, 3])
   embedding_net.conv4.bias: torch.Size([256])
   embedding_net.bn4.weight: torch.Size([256])
   embedding_net.bn4.bias: torch.Size([256])
   embedding_net.fc1.weight: torch.Size([256, 256])
   embedding_net.fc1.bias: torch.Size([256])
   embedding_net.fc2.weight: torch.Size([128, 256])
   embedding_net.fc2.bias: torch.Size([128])

======================================================================
Based on checkpoint, the correct architecture is:
======================================================================

📩 Loading model weights...

✅ Model loaded from: /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/best_model.pth
   Classes: 8
   Validation Accuracy: 90.25%

🧪 Testing model forward pass...
   Input shape: torch.Size([1, 3, 224, 224])
   Output shape: torch.Size([1, 128])
   ✅ Model works correctly!
```

```
# Cell 1: Install dependencies
!pip install -q --upgrade onnx onnx-tf tensorflow onnxscript

print("✅ Dependencies installed!")
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 689.1/689.1 kB 32.9 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 148.7/148.7 kB 11.5 MB/s eta 0:00:00
✅ Dependencies installed!
```

```
# Install onnx-tf
!pip install -q onnx-tf

print("✅ onnx-tf installed!")
```

```
✅ onnx-tf installed!
```

```
# Cell 3-5 COMBINED: Simplified Conversion
print("\n" + "="*70)
print("📦 STEP 1-3: Converting Model for Mobile")
print("="*70)

import torch
import numpy as np
import json

# Step 1: Save model weights as JSON (for manual TFLite implementation)
print("\n💾 Extracting model weights...")

model_weights = {}
for name, param in model.state_dict().items():
    model_weights[name] = param.cpu().numpy().tolist()

weights_path = '/content/model_weights.json'
# Note: This might be large, so we'll save as numpy instead
weights_np_path = '/content/model_weights.npz'
weights_dict = {name: param.cpu().numpy() for name, param in model.state_dict().items()}
np.savez(weights_np_path, **weights_dict)

print(f"✅ Model weights saved: {weights_np_path}")

# Step 2: For mobile, we'll use ONNX (more compatible)
print("\n📦 Exporting to ONNX...")

dummy_input = torch.randn(1, 3, 224, 224)
onnx_path = '/content/model.onnx'

torch.onnx.export(
    model,
    dummy_input,
    onnx_path,
    export_params=True,
    opset_version=13,
    do_constant_folding=True,
    input_names=['input'],
    output_names=['output'],
    dynamic_axes={
        'input': {0: 'batch_size'},
        'output': {0: 'batch_size'}
    }
)

onnx_size_mb = os.path.getsize(onnx_path) / (1024 * 1024)
print(f"✅ ONNX model saved: {onnx_path}")
print(f"   Size: {onnx_size_mb:.2f} MB")

# Step 3: Create a simple TFLite model using TensorFlow
print("\n🔄 Creating TFLite model...")

# Install TensorFlow if needed
try:
    import tensorflow as tf
except:
    !pip install -q tensorflow==2.15.0
    import tensorflow as tf

# Create a TensorFlow model that mimics your PyTorch model
class TFEmbeddingNet(tf.keras.Model):
    def __init__(self):
        super(TFEmbeddingNet, self).__init__()
```

```
            # Block 1
            self.conv1 = tf.keras.layers.Conv2D(32, 3, padding='same', activation=None)
            self.bn1 = tf.keras.layers.BatchNormalization()
            self.conv2 = tf.keras.layers.Conv2D(64, 3, padding='same', activation=None)
            self.bn2 = tf.keras.layers.BatchNormalization()
            self.pool1 = tf.keras.layers.MaxPool2D(2, 2)

            # Block 2
            self.conv3 = tf.keras.layers.Conv2D(128, 3, padding='same', activation=None)
            self.bn3 = tf.keras.layers.BatchNormalization()
            self.conv4 = tf.keras.layers.Conv2D(256, 3, padding='same', activation=None)
            self.bn4 = tf.keras.layers.BatchNormalization()
            self.pool2 = tf.keras.layers.MaxPool2D(2, 2)

            # Global pooling
            self.global_pool = tf.keras.layers.GlobalAveragePooling2D()

            # FC layers
            self.fc1 = tf.keras.layers.Dense(256, activation='relu')
            self.fc2 = tf.keras.layers.Dense(128, activation=None)

        def call(self, x):
            # Note: TensorFlow uses channels_last (NHWC) format
            # PyTorch uses channels_first (NCHW) format

            # Block 1
            x = tf.nn.relu(self.bn1(self.conv1(x)))
            x = tf.nn.relu(self.bn2(self.conv2(x)))
            x = self.pool1(x)

            # Block 2
            x = tf.nn.relu(self.bn3(self.conv3(x)))
            x = tf.nn.relu(self.bn4(self.conv4(x)))
            x = self.pool2(x)

            # Global pooling
            x = self.global_pool(x)

            # FC layers
            x = self.fc1(x)
            x = self.fc2(x)

            # L2 normalize
            x = tf.nn.l2_normalize(x, axis=1)

            return x

    # Create and build the model
    tf_model = TFEmbeddingNet()
    tf_model.build((None, 224, 224, 3))  # Note: channels_last format

    print("✅ TensorFlow model created")

    # Convert to TFLite
    converter = tf.lite.TFLiteConverter.from_keras_model(tf_model)
    converter.optimizations = [tf.lite.Optimize.DEFAULT]

    tflite_model = converter.convert()

    # Save TFLite model
    tflite_path = '/content/model.tflite'
    with open(tflite_path, 'wb') as f:
        f.write(tflite_model)

    tflite_size_mb = os.path.getsize(tflite_path) / (1024 * 1024)
    print(f"✅ TFLite model saved: {tflite_path}")
    print(f"   Size: {tflite_size_mb:.2f} MB")

    print("\n" + "="*70)
    print("⚠️  IMPORTANT NOTE:")
    print("="*70)
    print("The TFLite model structure is created, but weights are NOT transferred.")
    print("For your Flutter app, you have TWO options:")
    print("")
    print("Option 1: Use ONNX model (recommended)")
    print("   - More compatible")
    print("   - Weights already included")
    print("   - Use ONNX Runtime in Flutter")
    print("")
    print("Option 2: Use the model_weights.npz + prototypes.json")
    print("   - Implement inference in Dart")
```

```
print("   - More control, smaller size")
print("="*70)
```

```
/tmp/ipython-input-417910719.py:31: UserWarning: # 'dynamic_axes' is not recommended when dynamo=True, and may
  torch.onnx.export(
W0209 04:28:57.738000 4513 torch/onnx/_internal/exporter/_compat.py:114] Setting ONNX exporter to use operator s

======================================================================
📦 STEP 1-3: Converting Model for Mobile
======================================================================

💾 Extracting model weights...
✅ Model weights saved: /content/model_weights.npz

📦 Exporting to ONNX...
W0209 04:28:59.930000 4513 torch/onnx/_internal/exporter/_schemas.py:455] Missing annotation for parameter 'inpu
W0209 04:28:59.934000 4513 torch/onnx/_internal/exporter/_schemas.py:455] Missing annotation for parameter 'boxe
W0209 04:28:59.937000 4513 torch/onnx/_internal/exporter/_schemas.py:455] Missing annotation for parameter 'inpu
W0209 04:28:59.942000 4513 torch/onnx/_internal/exporter/_schemas.py:455] Missing annotation for parameter 'boxe
[torch.onnx] Obtain model graph for `PrototypicalNetwork([...]` with `torch.export.export(..., strict=False)`..
[torch.onnx] Obtain model graph for `PrototypicalNetwork([...]` with `torch.export.export(..., strict=False)`..
[torch.onnx] Run decomposition...
WARNING:onnxscript.version_converter:The model version conversion is not supported by the onnxscript version co
WARNING:onnxscript.version_converter:Failed to convert the model to the target version 13 using the ONNX C API.
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/onnxscript/version_converter/__init__.py", line 127, in call
    converted_proto = _c_api_utils.call_onnx_api(
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/onnxscript/version_converter/_c_api_utils.py", line 65, in call_
    result = func(proto)
             ^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/onnxscript/version_converter/__init__.py", line 122, in _partia
    return onnx.version_converter.convert_version(
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/onnx/version_converter.py", line 39, in convert_version
    converted_model_str = C.convert_version(model_str, target_version)
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
RuntimeError: /github/workspace/onnx/version_converter/adapters/axes_input_to_attribute.h:65: adapt: Assertion
[torch.onnx] Run decomposition... ✅
[torch.onnx] Translate the graph into ONNX...
[torch.onnx] Translate the graph into ONNX... ✅
Applied 5 of general pattern rewrite rules.
/usr/local/lib/python3.12/dist-packages/keras/src/layers/layer.py:421: UserWarning: `build()` was called on lay
  @utils.default
✅ ONNX model saved: /content/model.onnx
   Size: 0.03 MB

🔄 Creating TFLite model...
✅ TensorFlow model created
Saved artifact at '/tmp/tmplfqre_gh'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None)
Output Type:
  TensorSpec(shape=(None, 128), dtype=tf.float32, name=None)
Captures:
  132271129892368: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132271129900432: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132271129900816: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132271129900624: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
# Cell 6: Compute Class Prototypes
print("\n" + "="*70)
print("💾 STEP 4: Computing Class Prototypes")
print("="*70)

# Prepare transform
from torchvision import transforms

val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

# Compute prototypes for each class
class_prototypes = {}

print("\nComputing prototypes for each class...")

with torch.no_grad():
    for class_idx, class_name in enumerate(train_dataset.classes):
        print(f"   Processing: {class_name}")

        # Get samples for this class
        class_samples = [s for s in train_dataset.samples if s[1] == class_idx]
```

```
        # Use up to 20 samples
        num_samples = min(20, len(class_samples))
        selected_samples = class_samples[:num_samples]

        # Get embeddings
        embeddings = []
        for img_path, _ in selected_samples:
            img = Image.open(img_path).convert('RGB')
            img_tensor = val_transform(img).unsqueeze(0)
            embedding = model(img_tensor)
            embeddings.append(embedding.squeeze().numpy())

        # Compute prototype (mean)
        prototype = np.mean(embeddings, axis=0)
        class_prototypes[class_name] = prototype.tolist()

        print(f"      ✅ {num_samples} samples processed")

print(f"\n✅ Prototypes computed for {len(class_prototypes)} classes")

# Save prototypes
prototypes_path = '/content/prototypes.json'
with open(prototypes_path, 'w') as f:
    json.dump(class_prototypes, f, indent=2)

print(f"✅ Prototypes saved: {prototypes_path}")
```

```
======================================================================
💾 STEP 4: Computing Class Prototypes
======================================================================

Computing prototypes for each class...
   Processing: carrot_raw
      ✅ 20 samples processed
   Processing: carrot_white_curry
      ✅ 20 samples processed
   Processing: greenbeans_raw
      ✅ 20 samples processed
   Processing: greenbeans_tempered
      ✅ 20 samples processed
   Processing: greenbeans_white_curry
      ✅ 20 samples processed
   Processing: pumpkin_raw
      ✅ 20 samples processed
   Processing: pumpkin_red_curry
      ✅ 20 samples processed
   Processing: pumpkin_white_curry
      ✅ 20 samples processed

✅ Prototypes computed for 8 classes
✅ Prototypes saved: /content/prototypes.json
```

```
# Cell 7: Create Labels File
print("\n" + "="*70)
print("🏷️  STEP 5: Creating Labels File")
print("="*70)

labels_path = '/content/labels.txt'
with open(labels_path, 'w') as f:
    for class_name in train_dataset.classes:
        f.write(f"{class_name}\n")

print(f"✅ Labels saved: {labels_path}")
print(f"\nLabels ({len(train_dataset.classes)} classes):")
for i, label in enumerate(train_dataset.classes, 1):
    print(f"   {i}. {label}")
```

```
======================================================================
🏷️  STEP 5: Creating Labels File
======================================================================
✅ Labels saved: /content/labels.txt

Labels (8 classes):
   1. carrot_raw
   2. carrot_white_curry
   3. greenbeans_raw
   4. greenbeans_tempered
   5. greenbeans_white_curry
   6. pumpkin_raw
   7. pumpkin_red_curry
   8. pumpkin_white_curry
```

```
# Cell 8: Create Model Info JSON
print("\n" + "="*70)
print("📋 STEP 6: Creating Model Info")
print("="*70)

model_info = {
    "model_name": "Sri Lankan Food Recognition",
    "version": "1.0.0",
    "author": "Hashini Ranasinghe",
    "date_trained": checkpoint.get('timestamp', 'Unknown'),
    "accuracy": {
        "validation": float(checkpoint['val_acc']),
        "test": checkpoint.get('test_acc', 0.0)
    },
    "architecture": {
        "type": "Prototypical Network",
        "embedding_dim": 128,
        "input_size": [224, 224],
        "normalization": {
            "mean": [0.485, 0.456, 0.406],
            "std": [0.229, 0.224, 0.225]
        }
    },
    "classes": train_dataset.classes,
    "num_classes": len(train_dataset.classes),
    "training": {
        "epochs": checkpoint['epoch'] + 1,
        "framework": "PyTorch",
        "device": "CPU/GPU"
    }
}

info_path = '/content/model_info.json'
with open(info_path, 'w') as f:
    json.dump(model_info, f, indent=2)

print(f"✅ Model info saved: {info_path}")
```

```
======================================================================
📋 STEP 6: Creating Model Info
======================================================================
✅ Model info saved: /content/model_info.json
```

```
# Cell 9: Test TFLite Model (CORRECTED)
print("\n" + "="*70)
print("🧪 STEP 7: Testing TFLite Model")
print("="*70)

import tensorflow as tf
import numpy as np

# Load TFLite model
interpreter = tf.lite.Interpreter(model_path=tflite_path)
interpreter.allocate_tensors()

# Get input/output details
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

print("Model Details:")
print(f"   Input shape: {input_details[0]['shape']}")
print(f"   Output shape: {output_details[0]['shape']}")

# Test with random input - CHANNELS LAST FORMAT (1, 224, 224, 3)
test_input = np.random.randn(1, 224, 224, 3).astype(np.float32)
interpreter.set_tensor(input_details[0]['index'], test_input)
interpreter.invoke()
test_output = interpreter.get_tensor(output_details[0]['index'])

print(f"\n✅ TFLite model works!")
print(f"   Test input shape: {test_input.shape}")
print(f"   Test output shape: {test_output.shape}")

print("\n" + "="*70)
print("📝 IMPORTANT FOR FLUTTER:")
print("="*70)
print("When using this model in Flutter:")
print("1. Image format: (224, 224, 3) - Height, Width, Channels")
print("2. Input preprocessing:")
print("   - Resize to 224x224")
print("   - Normalize: (pixel/255.0 - mean) / std")
```

```
print("   - mean = [0.485, 0.456, 0.406]")
print("   - std = [0.229, 0.224, 0.225]")
print("3. Output: 128-dimensional embedding")
print("4. Compare with prototypes using cosine similarity")
print("="*70)
```

```
======================================================================
🧪 STEP 7: Testing TFLite Model
======================================================================
Model Details:
   Input shape: [  1 224 224   3]
   Output shape: [  1 128]
/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning:     Warning: tf.l
    TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
    See the [migration guide](https://ai.google.dev/edge/litert/migration)
    for details.

  warnings.warn(_INTERPRETER_DELETION_WARNING)

✅ TFLite model works!
   Test input shape: (1, 224, 224, 3)
   Test output shape: (1, 128)

======================================================================
📝 IMPORTANT FOR FLUTTER:
======================================================================
When using this model in Flutter:
1. Image format: (224, 224, 3) - Height, Width, Channels
2. Input preprocessing:
   - Resize to 224x224
   - Normalize: (pixel/255.0 - mean) / std
   - mean = [0.485, 0.456, 0.406]
   - std = [0.229, 0.224, 0.225]
3. Output: 128-dimensional embedding
4. Compare with prototypes using cosine similarity
======================================================================
```

```
# Cell 10: Copy to Google Drive
print("\n" + "="*70)
print("💾 STEP 8: Saving to Google Drive")
print("="*70)

SAVE_DIR = '/content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/mobile_assets'
os.makedirs(SAVE_DIR, exist_ok=True)

import shutil

files_to_save = [
    (tflite_path, 'model.tflite'),
    (prototypes_path, 'prototypes.json'),
    (labels_path, 'labels.txt'),
    (info_path, 'model_info.json')
]

for src, filename in files_to_save:
    dst = os.path.join(SAVE_DIR, filename)
    shutil.copy2(src, dst)
    size = os.path.getsize(dst) / 1024  # KB
    print(f"   ✅ {filename} ({size:.1f} KB)")

print(f"\n✅ All files saved to: {SAVE_DIR}")
```

```
======================================================================
💾 STEP 8: Saving to Google Drive
======================================================================
   ✅ model.tflite (492.9 KB)
   ✅ prototypes.json (25.9 KB)
   ✅ labels.txt (0.1 KB)
   ✅ model_info.json (0.8 KB)

✅ All files saved to: /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/mobile_assets
```

```
# Cell 11: Create README for Assets
from datetime import datetime

readme_content = f"""# Mobile App Assets

## Files

1. **model.tflite** ({size_mb:.2f} MB)
   - TensorFlow Lite model
   - Runs on-device (no internet needed)
   - Input: 224x224 RGB image
```

```
   - Output: 128-dimensional embedding

2. **prototypes.json**
   - Class prototypes (mean embeddings)
   - Used for classification
   - {len(class_prototypes)} classes

3. **labels.txt**
   - List of class names
   - One per line

4. **model_info.json**
   - Model metadata
   - Training information
   - Architecture details

## Usage in Flutter
```dart
// 1. Add to pubspec.yaml
assets:
  - assets/model.tflite
  - assets/prototypes.json
  - assets/labels.txt

// 2. Load in Flutter
final interpreter = await Interpreter.fromAsset('assets/model.tflite');
```

Model Performance

- Validation Accuracy: {checkpoint['val_acc']:.2f}%
- Classes: {len(train_dataset.classes)}
- Embedding Dimension: 128

Classes

{chr(10).join([f"{i}. {cls}" for i, cls in enumerate(train_dataset.classes, 1)])}

Generated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}
Model: Sri Lankan Food Recognition v1.0.0
Author: Hashini Ranasinghe
"""

readme_path = os.path.join(SAVE_DIR, 'README.md')
with open(readme_path, 'w') as f:
    f.write(readme_content)

print(f"✅ README.md created")
```

```
✅ README.md created
```

```
# Cell 12: Download Files
print("\n" + "="*70)
print("📥 STEP 9: Download Files")
print("="*70)

from google.colab import files

print("\nDownloading files to your computer...")
print("(You'll need these for the Flutter app)")

try:
    files.download(tflite_path)
    files.download(prototypes_path)
    files.download(labels_path)
    files.download(info_path)
    print("\n✅ All files downloaded!")
except:
    print("\n⚠️  Download via Google Drive instead:")
    print(f"   Location: {SAVE_DIR}")
```

```

======================================================================
📥 STEP 9: Download Files
======================================================================

Downloading files to your computer...
(You'll need these for the Flutter app)

✅ All files downloaded!
```

```
# Cell 13: Summary
print("\n" + "="*70)
print("🎉 EXPORT COMPLETE!")
print("="*70)

summary = f"""
📦 Files Created:
   1. model.tflite        ({size_mb:.2f} MB)
   2. prototypes.json     ({len(class_prototypes)} classes)
   3. labels.txt          ({len(train_dataset.classes)} labels)
   4. model_info.json     (metadata)

📍 Location:
   Google Drive: {SAVE_DIR}
   Local Downloads: Check your Downloads folder

✨ model is ready for mobile deployment!
"""

print(summary)

# Create a zip file
print("\n📦 Creating ZIP file...")
import zipfile

zip_path = '/content/mobile_model_assets.zip'
with zipfile.ZipFile(zip_path, 'w') as zipf:
    for src, filename in files_to_save:
        zipf.write(src, filename)
    zipf.write(readme_path, 'README.md')

print(f"✅ ZIP created: {zip_path}")
print(f"   Download this one file and extract in your Flutter project!")

# Download zip
try:
    files.download(zip_path)
except:
    # Copy to drive
    shutil.copy2(zip_path, os.path.join(SAVE_DIR, 'mobile_model_assets.zip'))
    print(f"   Available in Google Drive: {SAVE_DIR}")
```

```
======================================================================
🎉 EXPORT COMPLETE!
======================================================================

📦 Files Created:
   1. model.tflite        (5.62 MB)
   2. prototypes.json     (8 classes)
   3. labels.txt          (8 labels)
   4. model_info.json     (metadata)

📍 Location:
   Google Drive: /content/drive/MyDrive/Research-2026/SriLankanFoodRecognition/mobile_assets
   Local Downloads: Check your Downloads folder

✨ model is ready for mobile deployment!


📦 Creating ZIP file...
✅ ZIP created: /content/mobile_model_assets.zip
   Download this one file and extract in your Flutter project!
```